Programmation réseaux

Master 1 CSIA

IV Les Threads

- Le problème fondamental avec la structure du programe serveur déjà présenté est que cette solution est difficilement extensible (serveurs FTP et Web).

Bien qu'il soit relativement facil d'écrire du code qui traite chaque demande de connexion, qui arrive, et chaque nouvelle tache comme un processus séparé; quand le moment viendra où le serveur devra traiter un nombre élevé de connexions simultanées les performances s'écrouleront.

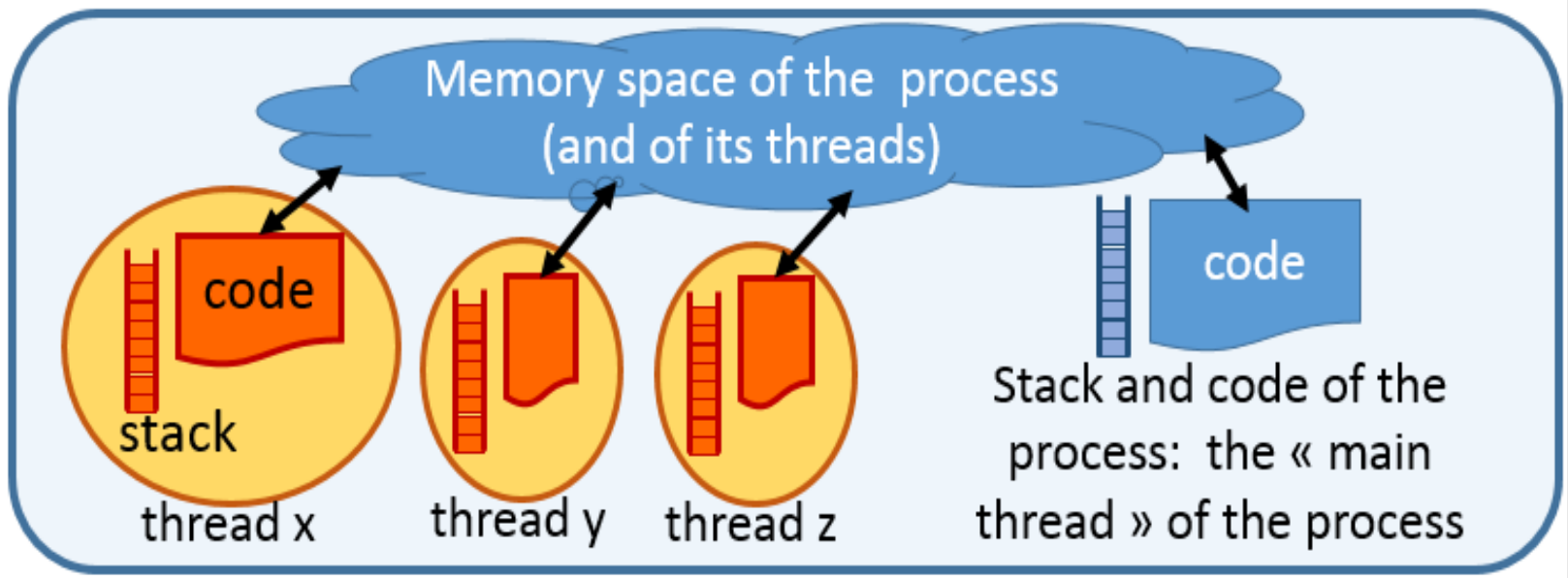
Il y a au moins deux solutions à ce problème.

1) La première solution est de réutiliser les processus au lieu d'en engendrer de nouveaux.

- Processus : L'entité engendrée et contrôlée par le système d'exploitation qui encapsule une application en cours d'exécution. Un processus a deux fonctions principales. La première est d'agir en tant que détenteur de ressources pour l'application, et la seconde est d'exécuter les instructions de l'application.

- Lorsque le serveur démarre, un nombre fixe de processus (disons 300) sont engendés pour traiter les requêtes.
- Les requêtes qui arrivent sont mises dans une file. Chaque processus retire une requête de la file, la sert, puis retire la requête suivante, etc.
- Il y toujours 300 processus séparés qui s'exécutent mais, parceque tout le travail supplémentaire pour construire et détruire les processus est évité, ces 300 processus peuvent maintenant faire le travail de 1000 processus.

2) La deuxième solution est d'utiliser des threads légers au lieu des processus lourds pour prendre en charge les connexions.



2) La deuxième solution est d'utiliser des threads légers

- Alors que chaque processus séparé a son prope bloc mémoire, les threads partagent le même epace mémoire.
- L'impact de l'exécution de plusieurs threads différents sur la machine du serveur est relativement minimal car tous les threads s'exécutent dans un processus.
- L'utilisation des threads apporte un gain de performance de l'ordre de trois par rapport à l'utilisation des processus.
- En combinant cela avec un pool de threads réutilisables le serveur peut devenir neuf fois plus rapide

Différences entre processus et threads (1)

- les processus sont plus lourds à créer que les threads
- les processus sont réellement indépendants
- les threads peuvent se partager des informations facilement
- les threads doivent eux-mêmes faire attention à ne pas" se marcher sur les pieds "

Différences entre processus et threads (2)

D'un point de vue programmation :

- pour passer un programme en multithread il faut avoir codé proprement et évité les variables globales
- pour passer un programme en multithread il faut faire attention aux accès en mémoire qui sont potentiellement partagés
- en multithread, fermer un fichier déjà ouvert le ferme pour tous les threads et pas seulement celui qui a appelé close()
- pour passer un programme en multiprocessus il suffit d'appeler fork
- en multiprocessus, fermer un fichier déjà ouvert ne le ferme que pour le fils qui a appelé close()

IV-1 Exécuter des threads en Java

Pour lancer un nouveau thread s'executant sur la machine virtuelle, on construit une instance de la classe Thread et on invoke sa méthode start(), comme ceci :

```
Thread t = new Thread();
t.start();
```

Pour donner à ce thread quelque chose à faire, on doit:

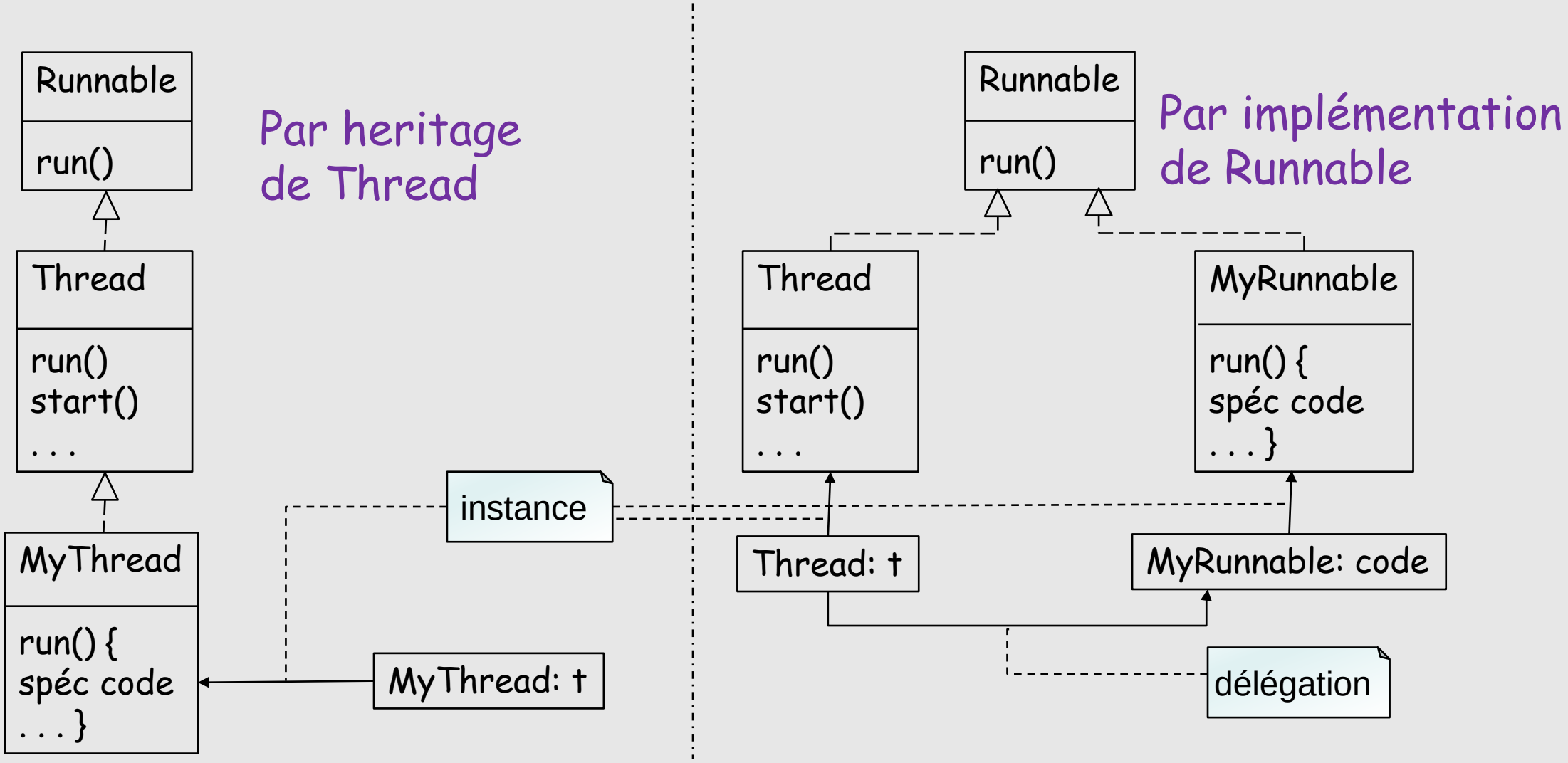
- soit créer une sous classe de Thread et surcharger sa méthode run()
- soit implémenter l'interface Runnable et passer l'objet Runnable au constructeur de Thread.

- Dans les deux cas, la clé c'est la méthode run(), qui a cette signature:

```
public void run()
```

- On va mettre tout le travail que le thread fait dans cette méthode.
- Cette méthode peut invoker d'autres méthodes; elle pourra construire d'autres objets; elle pourra même engendrer d'autres threads.
- Cependant, le thread commence là et s'arrête là aussi (dans run()).
- La méthode run() est au thread ce que la méthode main() est à un programme traditionnel.

IV-1.1 Création de threads



IV-1.1.1 Création par extension de la classe Thread

```
public class Multithread
{
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i=0; i<8; i++)
        {
            MthreadingDemo object = new MthreadingDemo();
            object.start();
        }
    }
}
```

```
class MthreadingDemo extends Thread {

    public void run()
    {
        try
        {
            // Affichage du thread qui s'exécute
            System.out.println ("Thread " +
                Thread.currentThread().getId() +
                " is running");

        }
        catch (Exception e)
        {
            // Renvoi d'une exception
            System.out.println ("Exception rencontrée");
        }
    }
}
```

IV-1.1.2 Création par implémentation de Runnable

```
public class Multithread
{
   public static void main(String[] args)
   {
      int n = 8; // Number of threads
      for (int i=0; i<8; i++)
      {
        Thread object = new Thread(new MthreadingDemo());
        object.start();
      }
   }
}
```

```
class MthreadingDemo implements Runnable {

   public void run()
   {
      try
      {
         // Affichage du thread qui s'exécute
         System.out.println ("Thread " +
              Thread.currentThread().getId() +
              " is running");

      }
      catch (Exception e)
      {
         // Renvoi d'une exception
         System.out.println ("Exception rencontrée");
      }
   }
}
```

IV-1.2 Différences entre Thread et Runnable dans Java

1. Chaque thread crée par extension de la classe Thread crée un objet unique ,pour lui, et il est associé avec cet objet. De l'autre côté, chaque thread crée par imlémentation de l'interface Runnable partage la même instance runnable.
2. Puisque chaque thread est associé à un objet unique lorsqu'il est crée par extension de la classe Thread, plus de mémoire est nécessaire. Alors que chaque thread crée par implementation de l'interface Runnable partage le même espace de l'objet donc, cela nécessite moins de mémoire.

3. Si on étend la classe Thread alors on ne pourra heriter aucune autre classe puisque Java ne permet pas l'heritage multiple alors que, implémenter Runnable donne une chance pour une classe d'heriter n'importe quelle autre.
4. On ne doit étendre la classe Thread que si on doit surchager ou spécialiser quelques autres méthodes de la classe Thread. On doit implementer l'interface Runnable si on veut spécialiser la méthode run() uniquement.

5. Etendre la classe Thread introduit un couplage fort dans le code puisque le code de Thread et le travail du thread est contenu dans la même classe. Implémenter l'interface Runnable introduit un couplage faible dans le code puisque le code de Thread est séparé du travail assigné au thread.

IV-2 Serveur multi-threads

Il suffit de déléguer l'exécution du service à un Thread dédié…

```
// le service
class Service implements Runnable {
public Socket maSocket;
Service(Socket s) {
this.maSocket = s;}
void run() {
// code du service…
maSocket.close();
}
}
```

```
// squelette de serveur
ServerSocket socketAttente;
socketAttente = new ServerSocket(PORT);
do { // établissement d'une connexion (attente bloquante)
Socket s = socketAttente.accept();
// la communication est désormais possible, création du service
Thread t = new Thread(new Service(s));
// on démarre l'exécution concurrente du service
t.start();
} while (true);
socketAttente.close();
```

Extension de la classeThread …

```
// le service
class Service extends Thread {
public Socket maSocket;
Service(Socket s) {
this.maSocket = s;}
void run() {
// code du service…
maSocket.close();
}
}
```

```
// squelette de serveur
ServerSocket socketAttente;
socketAttente = new ServerSocket(PORT);
do { // établissement d'une connexion (attente bloquante)
Socket s = socketAttente.accept();
// la communication est désormais possible, création du service
Service t = new  Service(s);
// on démarre l'exécution concurrente du service
t.start();
} while (true);
socketAttente.close();
```

IV-3 Notion de Réserve de Threads /1

- Dans la partie précédente, nous avons vu deux façons de lancer un thread.
- Dans la pratique, ces deux méthodes ne sont pas réellement utilisables (Fonctionnent pour les petites applications).
- La gestion d'un grand nombre de threads entraîne également une surcharge pour la machine Java, qui peut pénaliser les performances de l'ensemble de l'application.
- Il y a donc un équilibre à trouver entre un nombre de threads suffisant mais pas trop important.
- Il est raisonnable, de séparer la création et la gestion des threads du reste de l'application.

IV-3 Notion de Réserve de Threads /2

- Les objets qui encapsulent ces fonctions sont connus sous le nom d'exécuteurs.
- La plupart des implémentations de l'exécuteur dans java.util.concurrent utilisent des pools de threads, qui consistent en des threads travailleurs.
- Ces pools de *threads* gèrent un nombre en général fixé et borné de threads.
- Lorsque l'on a une tâche à lancer dans un nouveau *thread* , on la soumet à cette réserve, qui la prend en charge, et la fait exécuter par un *thread* disponible

IV-3 Notion de Réserve de Threads /3 Les pools de threads

- Un pool de threads est un modèle de programmation permettant de gérer automatiquement un pool de threads de travail.
- Le pool est responsable d'un nombre fixe de threads.
- Il contrôle le moment où les threads sont créés, par exemple juste à temps lorsqu'ils sont nécessaires.
- Il contrôle également ce que les threads doivent faire lorsqu'ils ne sont pas utilisés, par exemple en les faisant attendre sans consommer de ressources de calcul.
- Chaque thread du pool est appelé worker ou worker thread.

IV-3 Notion de Réserve de Threads /3

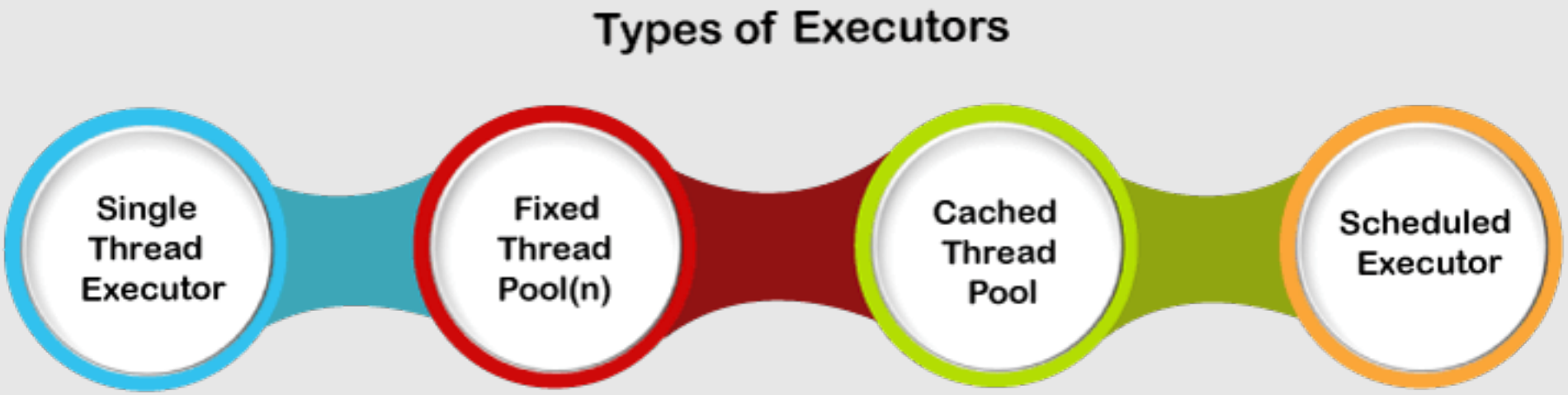
- Chaque worker est agnostique par rapport au type de tâches exécutées, tout comme l'utilisateur du pool de threads pour exécuter une suite de tâches similaires ou dissemblables (en termes de fonction appelée, d'arguments de fonction, de durée de tâche, etc.

- Les threads de travail sont conçus pour être réutilisés une fois la tâche terminée et offrent une protection contre l'échec inattendu de la tâche, comme la levée d'une exception, sans avoir d'impact sur le thread de travail lui-même.

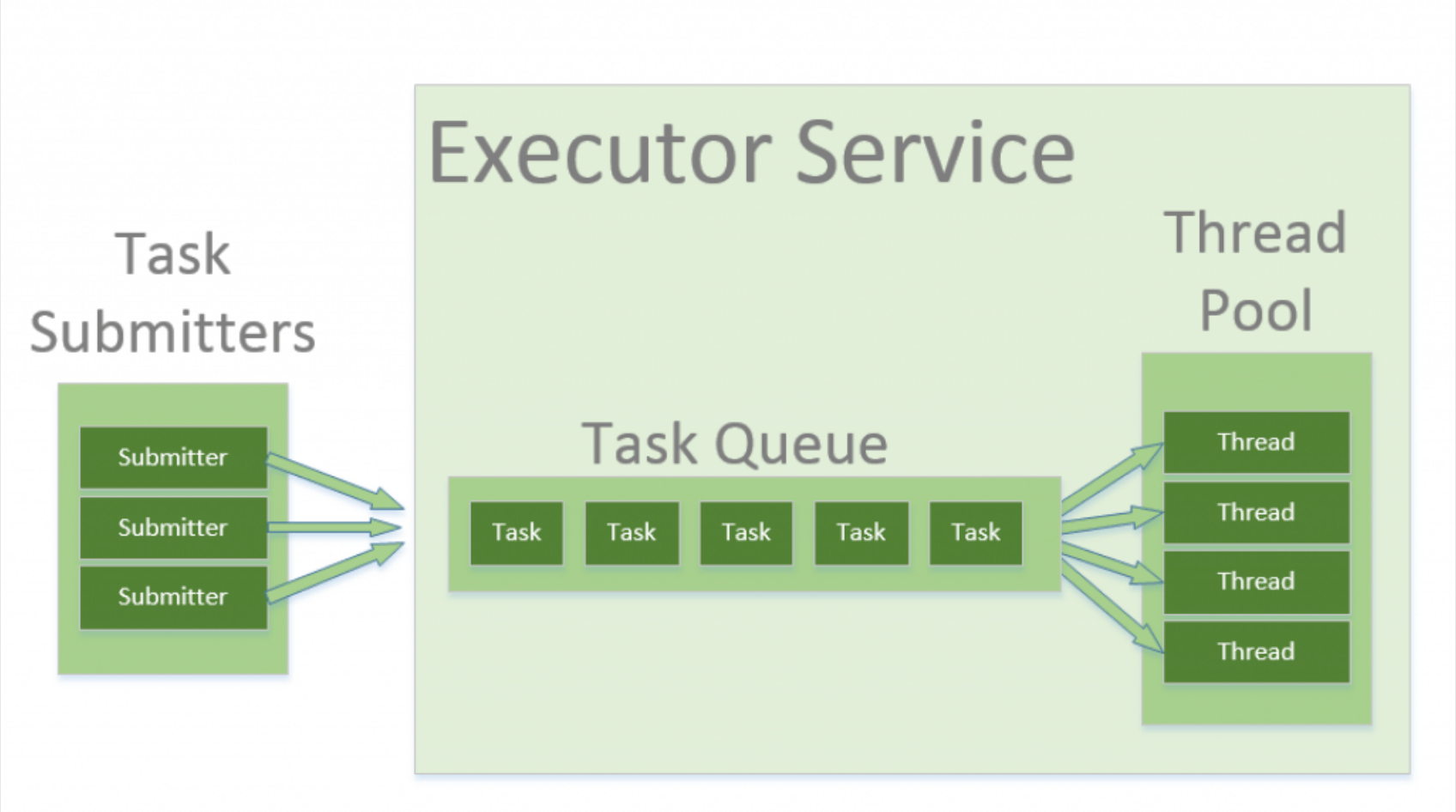
- Ceci est différent d'un thread unique qui est configuré pour l'exécution unique d'une tâche spécifique.

- Le pool peut fournir une certaine facilité pour configurer les threads de travail, par exemple en exécutant une fonction d'initialisation et en nommant chaque thread de travail à l'aide d'une convention de dénomination spécifique.

IV-3 Notion de Réserve de Threads /3



IV-3 Notion de Réserve de Threads / 4



IV-3 Notion de Réserve de Threads /5

```
package com.journaldev.threadpool;
public class WorkerThread implements Runnable {
    private String command;
    public WorkerThread(String s){
        this.command=s;
    }
     @Override
    public void run() {
        System.out.println(Thread.currentThread().getName()+" Start. Command = "+command);
        processCommand();
        System.out.println(Thread.currentThread().getName()+" End.");
    }
```

IV-3 Notion de Réserve de Threads / 6

```
private void processCommand() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public String toString(){
        return this.command;
    }
}
```

IV-3 Notion de Réserve de Threads / 7

```
• package com.journaldev.threadpool;
• import java.util.concurrent.ExecutorService;
• import java.util.concurrent.Executors;

• public class SimpleThreadPool {

•     public static void main(String[] args) {
•         ExecutorService  executor = Executors.newFixedThreadPool(5);
•         for (int i = 0; i < 10; i++) {
•             Runnable worker = new WorkerThread("" + i);
•             executor.execute(worker); }
•         executor.shutdown();
•         while (!executor.isTerminated()) {}
•         System.out.println("Finished all threads"); }}
```